

ArtsyPhartsy Bag Uploader

Technical Reference & Design Guide

May 2026

What this tool does

Watches three input folders for bag/apron photos, uses AI to detect and crop individual items, puffs or renders them with a clean studio background, then automatically creates draft WooCommerce products complete with names, descriptions, brand stories, pricing, SKUs, and images — finishing with an email notification.

Built for ArtsyPhartsy — artsyphartsy.org

1. Project Overview

The ArtsyPhartsy Bag Uploader is a Node.js automation tool running on WSL (Windows Subsystem for Linux) on a Windows 11 laptop. Someone photographs bags and aprons, drops the photos into input folders, and the tool handles everything else — AI detection, image enhancement, and WooCommerce listing creation.

1.1 Project Location

All source code lives at ~/artsy-phartsy/ on the WSL machine (jevdemon's laptop). The main files are:

File	Purpose
src/index.js	Folder watcher, queue manager, graceful shutdown
src/pipeline.js	Orchestrates the 4-step pipeline per photo
src/ai.js	All OpenAI API calls: detect, crop, puff, brand story
src/woocommerce.js	Name uniqueness checks, image upload, product creation
src/email.js	Notification email
src/logger.js	Winston logging to console and file
.env	All credentials — never committed to git
logs/uploader.log	Full run history for debugging

1.2 How to Run

Start the watcher from the artsy-phartsy directory:

```
npm start
```

Then drop a photo into one of the three input folders. The watcher picks it up automatically. Press Ctrl+C for graceful shutdown.

1.3 Input Folders and Pricing

Folder	Item Type	Price
input/regular/	Light fabric bags	\$20.00
input/heavy/	Heavy fabric bags	\$25.00
input/aprons/	Aprons	\$15.00

Tip

Prices are set in the .env file via PRICE_HEAVY, PRICE_LIGHT, and PRICE_APRON variables and can be changed without touching the code.

2. The Four-Step Pipeline

Every photo that enters an input folder goes through these four steps in sequence. If any step fails, the tool logs a warning and uses a fallback rather than crashing.

Step 1 — AI Item Detection (`detectBags` in `ai.js`)

The photo is sent to GPT-4o with a detailed prompt asking it to identify every bag or apron in the image and return:

- A creative "Miss ____" product name reflecting the fabric pattern and vibe
- A 1-2 sentence product description highlighting the upcycled fabric
- Normalized bounding box coordinates (x, y, width, height as values 0.0–1.0)

After detection, each name is checked against WooCommerce for uniqueness (`checkNameExists`). If taken, a whimsical variant is tried in order:

- "Miss X Too" → "Miss X's Sister" → "Miss X's Twin" → "Miss X's Cousin" → "Miss X's Doppelgänger" → "Miss X's Long Lost Cousin"
- If all variants are taken, GPT-4o is asked to generate a completely new name

Design Decision — GPT-4o for Detection

GPT-4o was chosen over simpler object detection models because it simultaneously identifies bags, names them creatively in the "Miss ____" style, writes product descriptions, and returns bounding boxes. A dedicated object detection model would require a separate naming/description step. The all-in-one approach saves API calls and produces more contextually appropriate names.

Step 2 — Smart Cropping (`cropBags` in `ai.js`)

Each detected bounding box is used to crop the individual bag from the batch photo. Two important features:

EXIF Auto-Rotation

`sharp().rotate()` is called before cropping to apply phone EXIF orientation data. Without this, bags photographed in portrait mode on iPhones appear sideways. This was a persistent bug that took several iterations to permanently fix.

Smart Per-Side Padding

Rather than applying a fixed padding percentage on all sides, the algorithm checks each neighbouring bag's bounding box and limits padding to 45% of the available gap. This prevents crops from bleeding into adjacent bags in multi-bag photos.

```
For each side: padding = min(maxPadding, gapToNeighbour * 0.45)
```

Design Decision — Smart Padding

Fixed padding (e.g. always 10%) worked well for single-bag photos but caused crops to bleed into neighbours in batch photos. The 45% gap rule means single bags get generous padding while tightly-packed bags get proportionally smaller padding — automatically, without any configuration.

Step 3a — Bag Puffing (puffBag in ai.js)

The cropped flat bag image is sent to gpt-image-2 with a prompt to:

- Fill the bag with volume as if stuffed with a pillow
- Keep all fabric patterns, colors, and textures exactly as they appear
- Preserve the "ARTSY PHARTSY / OLD MADE NEW" branding tag
- Show the bag upright on a clean neutral light gray background
- Use professional product photography style lighting

Critical Design Decision — gpt-image-2 vs gpt-image-1

The project started with gpt-image-1 which was incapable of preserving the branding tag text — it always invented garbled text like "STEELRUBEL" or "BAS MALN HBS" regardless of how explicitly the prompt specified the exact text. Hours were spent on blur-based workarounds. Switching to gpt-image-2 (released April 2026) solved the problem immediately — its reasoning capability means it actually understands and follows the tag preservation instruction. gpt-image-2 also produces more realistic puffing with natural handle positioning.

Step 3b — Apron Mannequin Rendering (renderApron in ai.js)

For aprons, instead of puffing, the image is sent to gpt-image-2 to place the apron on a professional faceless dress form mannequin. The tag restoration system (detectTag + replaceTag) runs after rendering to verify tag integrity.

Step 3c — Background Replacement (prepareSecondaryImage in ai.js)

The original flat crop (before puffing) is sent to gpt-image-2 to replace the background with a clean neutral light gray studio backdrop matching the puffed image. This gives both product images a consistent, professional look regardless of what backdrop used when photographing.

Design Decision — Two Images Per Product

The primary image is the AI-puffed version showing the bag with volume and professional studio presentation. The secondary image is the background-replaced flat crop showing the real fabric, texture, and tag exactly as they are. This gives customers both a flattering product shot and an honest detail view.

Step 4 — WooCommerce Upload (woocommerce.js)

Each processed bag is uploaded as a draft product with:

- Both images uploaded to the WordPress media library first
- 1500ms delay after primary image upload (race condition fix — WordPress needs time to process the image before it can be attached to a product)
- Retry logic for timeouts: 3 attempts with 3s/6s/9s progressive delays
- Product created as draft with name, description, short description (brand story), price, SKU, category (ID 22 = Newest Products), and both images

3. Key Files in Detail

3.1 ai.js — All AI Operations

This is the largest and most complex file. It exports these functions:

Function	What it does
detectBags()	GPT-4o: finds bags in photo, names them, returns bounding boxes
cropBags()	sharp: EXIF rotate, smart-pad crop each bag from batch photo
generateAlternativeName()	GPT-4o: generates new name when all whimsical variants are taken
generateBrandStory()	GPT-4o: creates varied brand story text for each bag
detectTag()	GPT-4o: finds the "ARTSY PHARTSY" tag in an image, returns bbox
replaceTag()	sharp: composites original tag onto rendered image if AI garbled it
puffBag()	gpt-image-2: transforms flat bag into 3D puffed product shot
renderApron()	gpt-image-2: places apron on dress form mannequin
prepareSecondaryImage()	gpt-image-2: replaces background with clean studio gray
generateSKU()	Generates unique SKU: AP-XXXXXX-YYYYZZ format

3.2 woocommerce.js — Store Integration

Function	What it does
checkNameExists()	Searches WooCommerce for exact name match (any status)
ensureUniqueName()	Runs whimsical variant chain until unique name found
uploadImage()	Uploads to WordPress media library with auto-compression and retry
createDraftProduct()	Creates WooCommerce draft with full product data + brand story HTML
uploadBag()	Orchestrates image uploads then product creation for one bag
getDraftsUrl()	Returns WP Admin URL for the drafts listing page

3.3 pipeline.js — The Orchestrator

`processPhoto()` runs the four steps in sequence for a single photo file. It handles the batch directory creation, calls each step with appropriate arguments, assembles the final bag object, and moves the source photo to `processed/done/` or `processed/failed/` when complete.

3.4 index.js — The Watcher

Uses chokidar to watch all three input folders simultaneously. Processes files one at a time through a queue to avoid overwhelming the OpenAI API. Handles graceful shutdown on Ctrl+C by waiting for the current job to finish before exiting.

4. Environment Variables (.env)

All credentials and configuration live in `~/artsy-phartsy/.env`. This file must never be committed to git.

Variable	Purpose
OPENAI_API_KEY	OpenAI API key for GPT-4o and gpt-image-2
WOO_URL	Store URL: https://artsyphartsy.org
WOO_CONSUMER_KEY	WooCommerce REST API consumer key (ck_...)
WOO_CONSUMER_SECRET	WooCommerce REST API consumer secret (cs_...)
WP_USERNAME	WordPress username: XXXXXXXXXXXXX
WP_APP_PASSWORD	WordPress Application Password (no spaces) for media upload
SMTP_HOST	Email server: smtp.gmail.com
SMTP_PORT	Email port: 587
SMTP_USER	Gmail address for sending notifications
SMTP_PASS	Gmail App Password (not regular Gmail password)
NOTIFY_EMAILS	Comma-separated notification recipients
PRICE_HEAVY	Heavy bag price (default: 25)
PRICE_LIGHT	Light bag price (default: 20)
PRICE_APRON	Apron price (default: 15)
INPUT_FOLDER	Root input folder path (default: ./input)
OUTPUT_FOLDER	Where processed files are saved (default: ./output)
FILE_SETTLE_MS	Milliseconds to wait after file detected (default: 2000)

Authentication Note

WooCommerce API uses WOO_CONSUMER_KEY + WOO_CONSUMER_SECRET via Basic Auth header. WordPress media upload uses a separate WP Application Password (Settings → Users → Application Passwords in WP Admin) — NOT the WooCommerce API keys. These are two different authentication systems for two different APIs on the same WordPress install.

5. Key Design Decisions & Rationale

5.1 Why Node.js?

The partner system calling this tool is PHP-based. Node.js was chosen over Python because it offers better async/await handling for the multi-step AI pipeline, excellent sharp library for image processing, and straightforward npm ecosystem for the required dependencies (OpenAI, axios, chokidar, nodemailer, FormData, Winston).

5.2 Why WSL Instead of a Server?

The workflow involves photographing bags with her phone and immediately processing them. Running on her laptop via WSL means no network uploads of source photos — the phone photos drop directly into the watched folder via OneDrive or USB. A remote server would require an additional upload step.

5.3 Why Draft Products, Not Published?

The AI occasionally makes mistakes — wrong name, wrong price folder, a batch photo that didn't separate cleanly. Creating drafts means the owner reviews each listing before it goes live, enabling a quality check without requiring the, to do any of the tedious data entry. They can quickly scan the drafts, make minor adjustments, and publish.

5.4 WooCommerce Category ID 22

"Newest Products" category is hardcoded as ID 22. This is a deliberate simplification — all new bags go into the newest products category automatically. If the category ID ever changes, update the single reference in `createDraftProduct()` in `woocommerce.js`.

5.5 The Image Upload Race Condition

When an image is uploaded to WordPress, the API returns a media ID immediately but WordPress continues processing the image in the background. If a product is created too quickly, WordPress rejects the media ID as invalid because processing isn't complete. A 1500ms delay after each image upload resolved this reliably.

5.6 The Tag Text Problem

The biggest challenge of the project was preserving the "ARTSY PHARTSY / OLD MADE NEW" branding tag text during AI image transformation. gpt-image-1 consistently generated garbled text regardless of prompt instructions. The progression of attempted solutions was:

- Stronger prompt instructions — did not help
- Explicit text content in prompt ("The tag reads ARTSY PHARTSY...") — did not help
- Tag detection + compositing original tag back — partially worked but unreliable
- Blur the tag region — worked but inconsistent positioning
- Fixed blur zone — more reliable but still sometimes missed the tag
- Switch to gpt-image-2 — solved immediately due to reasoning capability

Lesson Learned

gpt-image-1 is a generative model — it generates plausible-looking pixels without truly understanding text preservation instructions. gpt-image-2's reasoning step means it actually processes the instruction contextually before generating. For any task requiring precise instruction following (especially text preservation), gpt-image-2 is significantly more reliable.

5.7 Photo Tips for Best Results

These tips are also in the project README:

- Leave 2-3 inches of backdrop between bags in batch photos for clean separation
- Leave space above the handles and below the bag bottom
- Maximum 3-4 bags per batch photo for best AI bounding box accuracy
- Good even lighting is more important than background choice
- Natural daylight near a window (overcast) gives the best results
- Avoid direct flash or single-side lighting — creates harsh shadows
- Background replacement handles any backdrop, but good bag/background contrast helps edge detection

6. Known Limitations & Pending Items

6.1 Multi-Bag Photo Separation

When bags are photographed very close together with minimal gap between them, GPT-4o's bounding boxes may overlap slightly, causing crops to bleed into neighbouring bags. The smart padding algorithm mitigates this but cannot fully compensate for truly touching bags. Solution: photograph batch photos with 2-3 inches of backdrop visible between each bag.

6.2 Apron Testing

The apron pipeline (mannequin rendering + tag restoration via `replaceTag/detectTag`) was implemented but not fully end-to-end tested during development. It should work but someone should run a test apron photo and verify the result before processing a large backlog of aprons.

6.3 Database Connection Errors

Occasionally WordPress returns "Error establishing a database connection" (HTTP 500). This is a hosting-side issue unrelated to the code. Affected photos are moved to `processed/failed/` and can be reprocessed by copying them back to the appropriate input folder.

6.4 Brand Story Variation

The AI varies brand story text per bag but may occasionally produce very similar wording across consecutive bags. The prompt encourages variation but results depend on the model's creativity in the moment.

6.5 OpenAI Costs

Each bag processed uses approximately:

- 1x GPT-4o call for detection + naming (~\$0.01)
- 1x gpt-image-2 call for puffing (~\$0.03-0.04)
- 1x gpt-image-2 call for background replacement (~\$0.03-0.04)
- 1x GPT-4o call for brand story (~\$0.005)

Total: approximately \$0.07-0.10 per bag processed. For a backlog of 100 bags, expect approximately \$7-10 in API costs.

7. Troubleshooting

7.1 Checking the Log

The log is the first place to look when something goes wrong:

```
tail -50 ~/artsy-phartsy/logs/uploader.log
```

Or search for errors on a specific bag:

```
grep "Miss Bag Name" ~/artsy-phartsy/logs/uploader.log
```

7.2 Reprocessing a Failed Photo

Photos that failed are in:

```
~/artsy-phartsy/input/*/processed/failed/
```

Copy them back to the appropriate input folder to retry:

```
cp ~/artsy-phartsy/input/heavy/processed/failed/photo.jpg ~/artsy-phartsy/input/heavy/
```

7.3 Common Errors and Fixes

Error	Fix
unsupported mimetype (application/octet-stream)	The tmp PNG path regex failed. Check that croppedPath has a standard extension.
ETIMEDOUT on image upload	Automatic retry fires up to 3 times. If all fail, reprocess the photo.
Error establishing a database connection	WordPress hosting issue. Wait a few minutes and reprocess.
#xxx is an invalid image ID	Race condition — WordPress still processing. The 1500ms delay should prevent this; if it recurs, increase the delay.
Name check failed — assuming unique	WooCommerce API timeout. The name check returns false (assumes unique) and continues. Check if the name already exists manually.
Puffing failed — using cropped original	OpenAI API error. Product is created with the flat crop instead of puffed image. Reprocess if puffed image is needed.

7.4 Starting Fresh

If the watcher gets into a bad state, restart it:

```
Ctrl+C (wait for graceful shutdown)
npm start
```

8. Dependencies

Package	Version	Purpose
openai	latest	GPT-4o + gpt-image-2 API client
sharp	latest	Image processing: rotate, crop, resize, compress, composite
chokidar	latest	Cross-platform file system watcher
axios	latest	HTTP client for WooCommerce and WordPress REST APIs
form-data	latest	Multipart form data for WordPress media upload
nodemailer	^8.0.8	Email notifications (updated for security)
winston	latest	Structured logging to console and file
dotenv	latest	Loads .env file into process.env

Node.js Version

The project requires Node.js 20+ installed via nvm on WSL. The system must use the WSL Linux node binary — NOT the Windows node.exe on the PATH. If sharp fails to install or run, verify which node is active with: `which node`

9. Quick Reference Card

Daily Operations

- Start: `cd ~/artsy-phartsy && npm start`
- Drop photos into `input/regular/`, `input/heavy/`, or `input/aprons/`
- Watch the log: `tail -f ~/artsy-phartsy/logs/uploader.log`
- Email arrives when drafts are ready — review and publish in WP Admin
- Stop: `Ctrl+C` (waits for current job to finish)

Photography Tips for Best Results

- Bright even lighting — overcast daylight near a window is ideal
- For batch photos: leave 2-3 inches of backdrop between each bag
- Leave space above handles and below bag bottom in the frame
- Maximum 3-4 bags per batch photo for accurate separation
- Background doesn't matter much — AI replaces it. But avoid same color as bag.

AI Models Used

- GPT-4o — bag detection, naming, brand story, tag detection
- gpt-image-2 — puffing, apron rendering, background replacement (reasoning model, follows instructions precisely)